## El proyecto

Crearemos un nuevo proyecto con dos clases:

**ClasePruebas.java**

```
import java.util.LinkedList;
import java.util.List;

public class ClasePruebas {
       private double suma;
    private List<Double> lista;

    public ClasePruebas() {
        System.out.println("construyendo clase");
    }
    public double echaCuentas() {
        lista = new LinkedList<Double>();
        for (int i = 0; i<100;i++){
            lista.add((double)i);
        }
        suma = 0.0;
        for (Double valor : lista){
            suma +=valor;
        }
        return suma;
    }

}
```

**Principal.java**

```
public class Principal {
       public static void main(String[] args) {
              ClasePruebas clase = new ClasePruebas();
              double valor = clase.echaCuentas();
              System.out.println("Sale " + valor);
       }
}
```

## Los breakpoints

Un Breakpoint es un punto en nuestro código en el que al ejecutar con debugger, la ejecución se detendrá y podremos ver los valores que tienen las variables en ese momento, modificarlos, etc, etc.

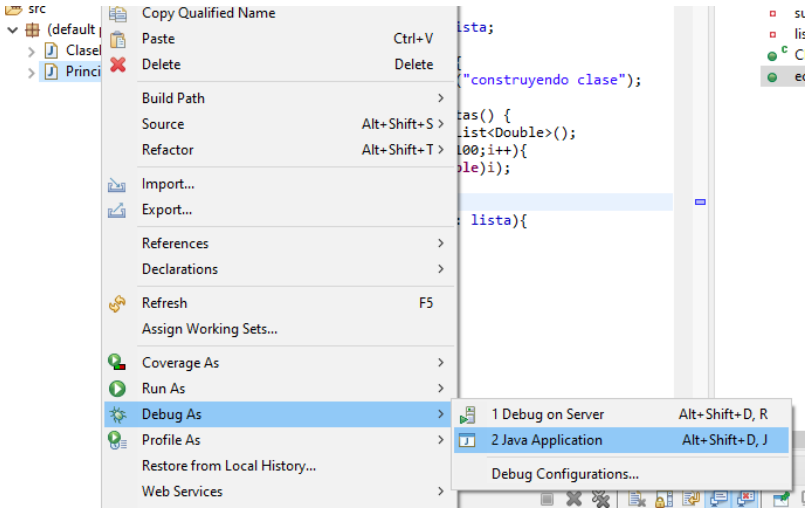
Poner un BreakPoint es sencillo, basta ir a nuestra clase y hacer doble clic en el margen izquierdo del editor, justo en la línea donde queremos que la ejecución se detenga. En la figura vemos un circulito a la izquierda de la línea 16, ese es el breakpoink que acabamos de poner haciendo doble clic justo ahí.

Si queremos quitarlo, nos basta con hacer doble clic en él nuevamente.

```
1  import java.util.LinkedList;
2  import java.util.List;
3
4  public class ClasePruebas {
5      private double suma;
6      private List<Double> lista;
7
8      public ClasePruebas() {
9          System.out.println("construyendo clase");
10     }
11     public double echaCuentas() {
12         lista = new LinkedList<Double>();
13         for (int i = 0; i<100;i++){
14             lista.add((double)i);
15         }
16         suma = 0.0;
17         for (Double valor : lista){
18             suma +=valor;
19         }
20         return suma;
21     }
22
23 }
24
```

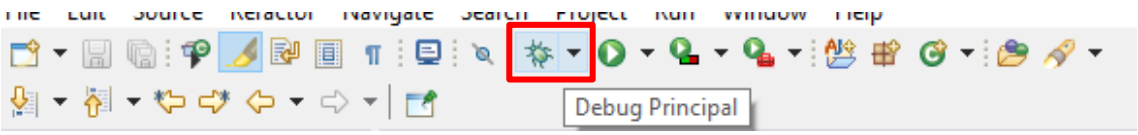
## Ejecutar la aplicación en modo debugger

Para arrancar el programa en modo debugger, sacamos el menú con el botón derecho del ratón sobre la clase que tiene el main. Buscamos "debug as" -> "java application"




Esto arrancará nuestro programa en modo debugger y se ejecutará de forma normal hasta que la ejecución pase por un Breakpoint, en ese momento se detendrá.

Este proceso del menú y "debug as" -> "Java application" sólo es necesario la primera vez. Las siguientes veces que queramos ejecutar, podemos usar el botón en la parte superior con el dibujo de una cucaracha (un "bug")



# Vista debugger

Cuando arranca el programa en modo debug, eclipse posiblemente nos pregunte si queremos pasar a la vista de debug. Le indicamos que sí, marcando el check de "Remember my decision" si no queremos que nos lo pregunte cada vez que arranquemos el debugger sin estar en esa vista.

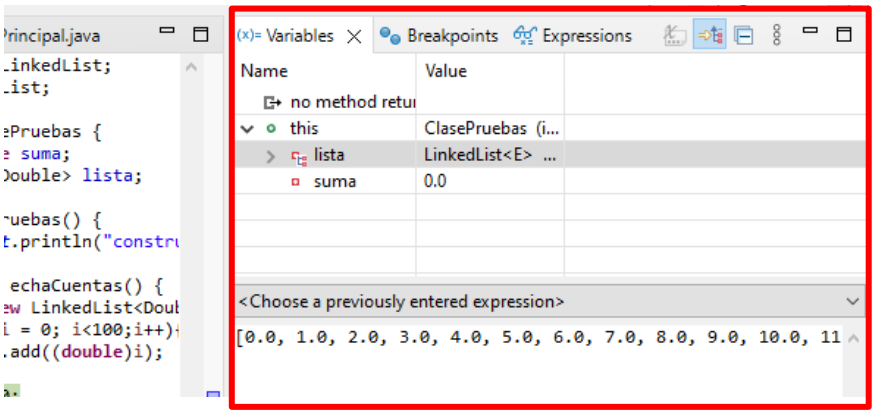
## Botones controlar la ejecución

En la vista de debugger tenemos botones para ir ejecutando paso a paso a partir del breakpoint donde nos hemos parado. Están en la parte superior y son como unas flechitas variadas amarillas

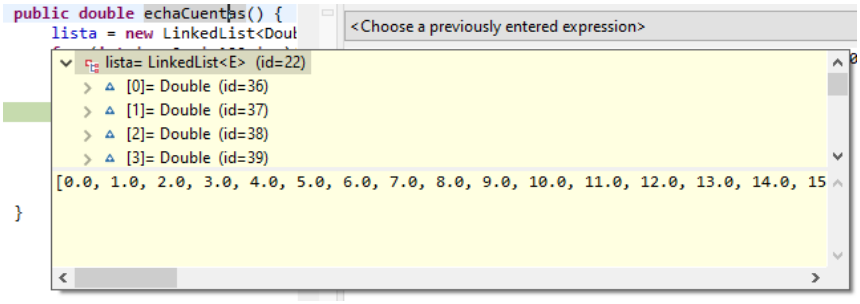
1. La flecha verde parecido al "play" hace que continue la ejecución de nuestro programa hasta el siguiente Breakpoint.
2. al botón de pausa suspende la ejecución del programa que está corriendo. Es una forma de parar "al vuelo" el programa, esté donde esté en ese momento, haya o no breakpoint.
3. El cuadrado rojo es el botón de stop, aborta la ejecución de nuestro programa.
4. Este botón no lo utilizaremos
5. El siguiente botón, una flecha amarilla que se "mete" entre dos puntos negros, imagina que nuestro programa tiene el breakpoint en una línea en la que se hace una llamada a un método y se ha parado ahí. Pulsando este botón, el debugger se meterá dentro del método y se parará en la primera línea del mismo, de forma que podemos depurar dentro del método
6. El siguiente botón, una flecha amarilla que salta sobre un punto negro, avanza un paso la ejecución del programa, pero sin meterse dentro del método que encuentre.
7. El último botón es una flecha saliendo de entre dos puntos negros, hace avanzar el programa hasta que nos salimos del método actual y vamos hasta el sitio donde ha sido llamado.

# Visualizar y modificar variables

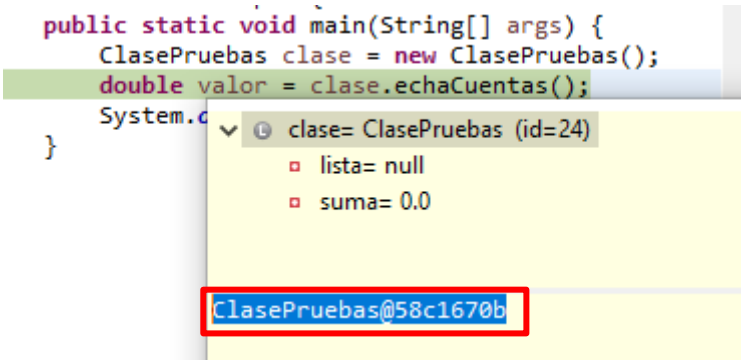
Tenemos una ventana etiquetada Variables. En ella podemos ver todas las variables definidas en el punto en el que está parado nuestro código. Es la de la siguiente figura



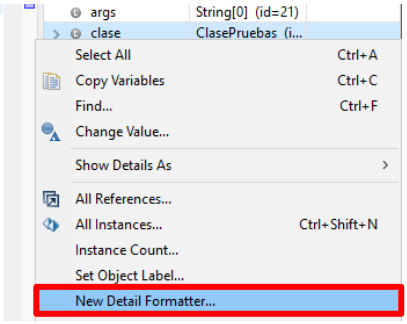
Vemos que en ese punto de ejecución nuestra lista ya tiene sus 100 elementos, pero suma todavía vale 0.0. La lista se ve un poco "rara" ya que estamos viendo todos los detalles internos de la clase LinkedList que hemos usado. Hay un botón que si lo pulsamos nos permite ver las colecciones de una forma más amigable, es el que se muestra en la siguiente figura



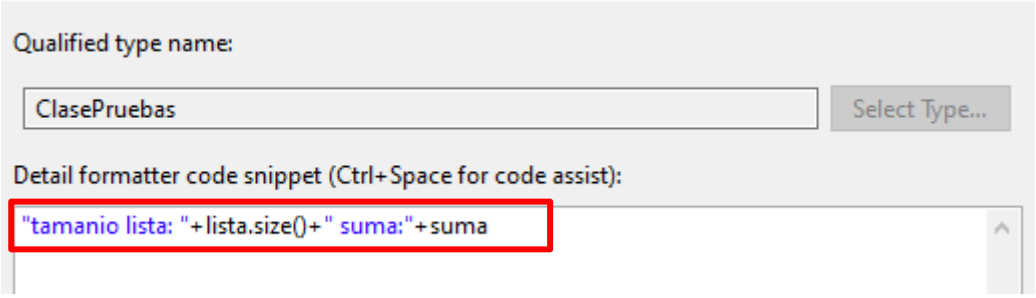

Cuando se muestra el valor de una variable, se llama a su método toString(), por lo que a veces la variable nos puede quedar algo fea y veremos algo como lo de la siguiente figura para la variable "clase". No es más que una llamada al método toString() de esa clase y lo que sale es lo que muestra



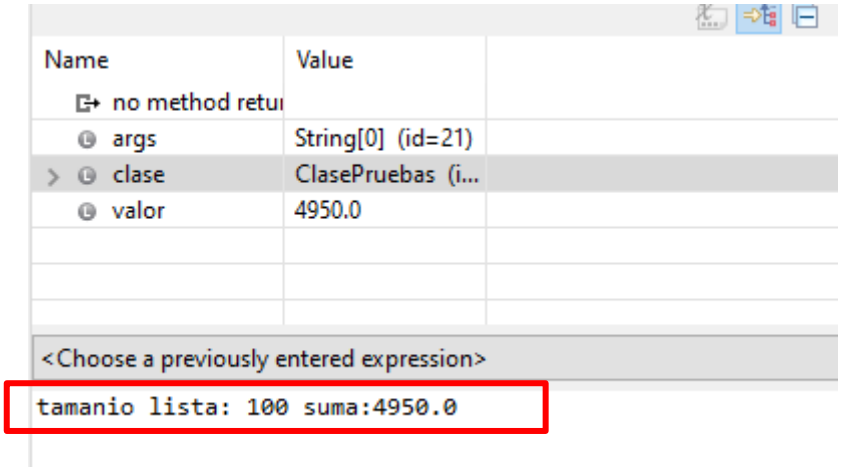

Vemos que sale el nombre de la clase, seguido de una @ y un número. Esto no nos dice nada. Sin embargo, podemos decirle a eclipse cómo queremos que se visualice una clase. Vamos a la ventana donde tenemos las variables, sacamos el menú con el botón derecho del ratón sobre la variable que nos interese y elegimos "New detail formatter"



Esto nos abrirá una ventana con una caja de texto donde podemos poner lo que queramos que se muestre para visualizar esa variable. Por ejemplo, en nuestra clase podemos querer que se vea el tamaño de la lista y el valor de la suma, así que nos basta con poner algo como lo de la siguiente figura

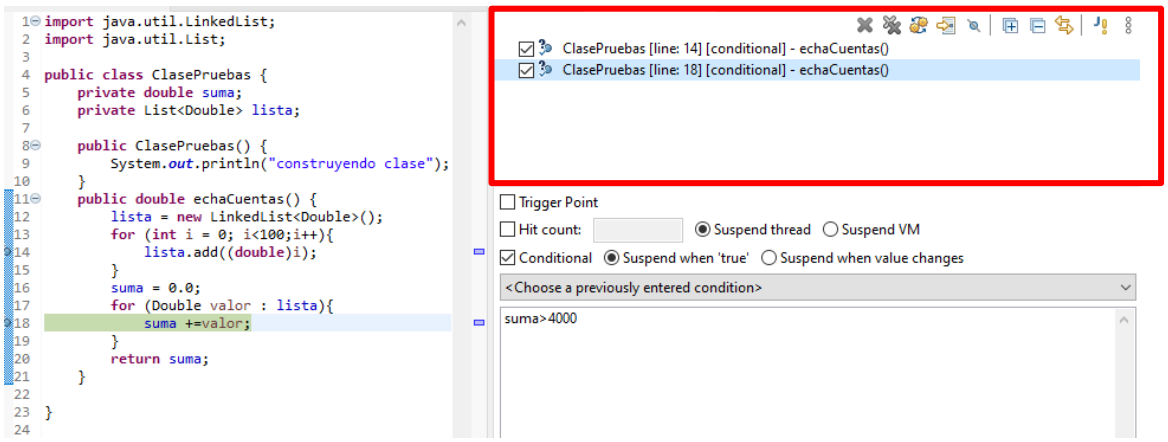



Ahora ya podremos ver la información de la clase siguiendo lo indicado anteriormente.

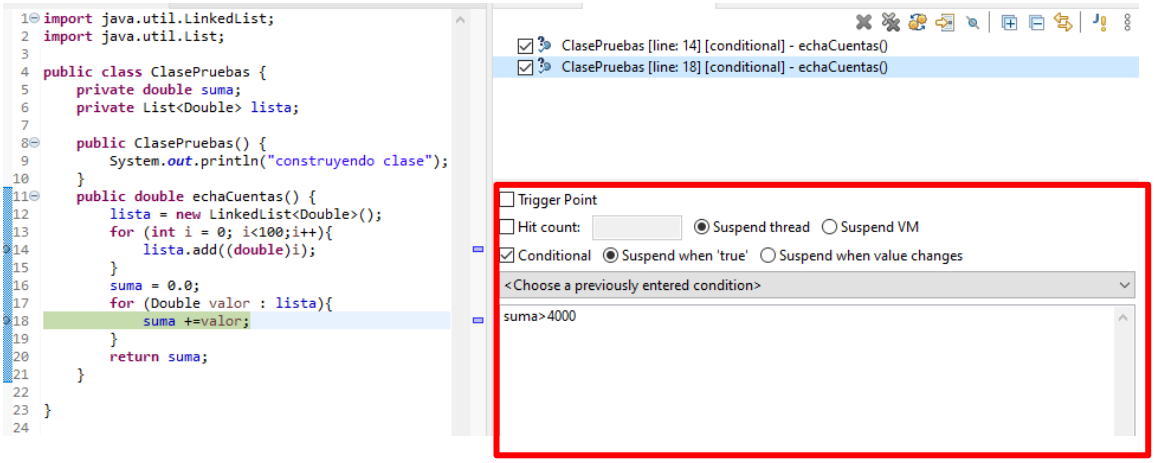



## Breakpoints condicionales

Tenemos otra ventana más con todos los breakpoints. Ahí podemos habilitarlos o deshabilitarlos sin más que marcar/desmarcar el check.



Lo que es más interesante es lo que se ve en la parte de debajo la ventana y es que podemos hacer que el código se pare o no en el breakpoint según alguna condición. Por ejemplo, ponemos el breakpoint en medio de un bucle, por ejemplo en el bucle donde estamos llenando la lista. Podemos hacer que el código se pare, por ejemplo, cuando estemos en la iteración 50 del bucle (cuando suma>4000), de esta forma




## Referencia

http://chuwiki.chuidiang.org/index.php?title=Empezando_con_el_debugger_de_eclipse